

Control Statements

Dr. Sanjit Kumar Saha



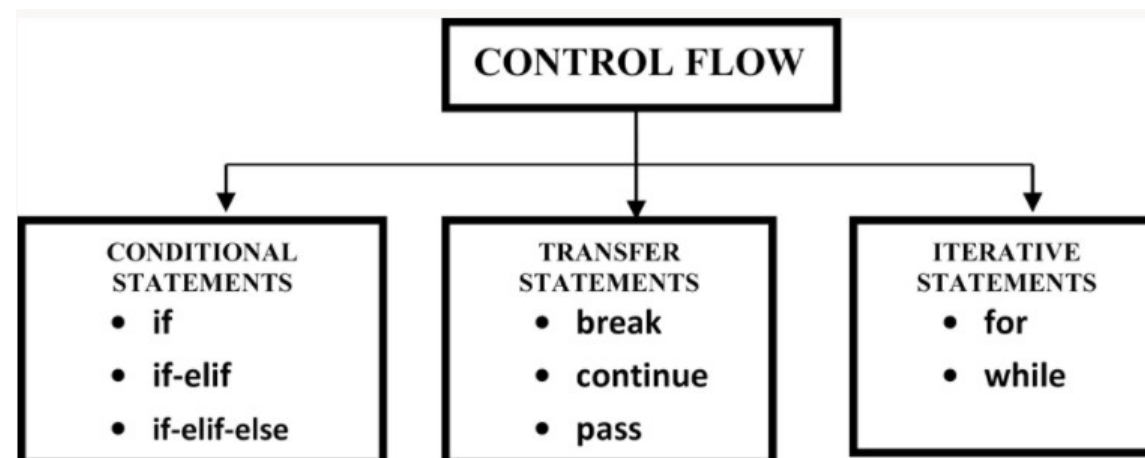
Gisma
University
of Applied
Sciences





Control Statements

- The code is always executed from the first to the last line of the program
 - Known as sequential control flow
- A control statement identifies the controlled circulation of set-of-instructions in which they will execute
 - There are three basic methods of control movement in it





Conditional Statements

- Conditional statements are used to control the flow of execution within a program
 - Based on some condition
- These conditions represent some choice point that will be evaluated to True or False
 - *if $x > 10$*



if Statement

- The if statement evaluates a condition (an expression that results in True or False).
 - If the condition is **True**, the code block *inside* the if statement is **executed**
 - If the condition is **False**, the code block is **skipped**

```
if <condition>:  
    statement
```

```
if x > 0:  
    → print("x is positive!")
```

- The **indentation** is very **important** in Python
 - The layout of your program is part of your program
 - Unlike language such as Java and C#
 - All lines indented to the same level after the if statement will automatically be part of the if statement
 - The **:** character is key in separating out the conditional part of the if statement from the statements



Comparison Operators

[Hunt 2019]

https://www.w3schools.com/python/python_operators.asp

5

- Python supports the usual logical conditions from mathematics:

Operator	Name	Example
<code>==</code>	Equal	<code>x == y</code>
<code>!=</code>	Not equal	<code>x != y</code>
<code>></code>	Greater than	<code>x > y</code>
<code><</code>	Less than	<code>x < y</code>
<code>>=</code>	Greater than or equal to	<code>x >= y</code>
<code><=</code>	Less than or equal to	<code>x <= y</code>

- These conditions can be used in several ways, most commonly in "if statements" and loops.
- Comparison operators return Boolean values



Logical Operators

- Logical operators are used to combine conditional statements
 - To create more complex conditions

Operator	Description	Example
<i>and</i>	Returns True if both statements are true	$x > 5$ <i>and</i> $x < 10$
<i>or</i>	Returns True if one of the statements is true	$x < 5$ <i>or</i> $x > 10$
<i>not</i>	Reverse the result, returns False if the result is true	$\text{not}(x > 5 \text{ and } x < 10)$

```
a = 200
```

```
b = 33
```

```
c = 500
```

```
if a > b and c > a:
```

```
    print("Both conditions are True")
```



if Statement

```
a = 33  
b = 200  
if b > a:  
    print("b is greater than a")
```

Output?



if Statement

- Multiple statements inside an *if* block
- All statements must be indented at the same level

```
age = 20
if age >= 18:
    print("You are an adult")
    print("You can vote")
    print("You have full legal rights")
```




if Statement

- Boolean variables can be used directly in if statements without comparison operators

```
is_logged_in = True  
if is_logged_in:  
    print("Welcome back!")
```



Else in an *If* Statement

- Else is an optional element
 - That can be run if the conditional part of the if statement returns False

```
if x > 0:  
    print("x is positive!")  
else:  
    print("x is not positive!")
```



Else in an *If* Statement

- The else statement provides a default action when none of the previous conditions are **true**
- Think of it as a "*catch-all*" for any scenario not covered by *if* and *elif* statements

```
number = 7

if number % 2 == 0:
    print("The number is even")
else:
    print("The number is odd")
```



Elif in an *If* Statement

- We may have several conditions to test
 - This *else-if* scenario is supported in Python by the *elif* element of an *if* statement
 - Use *elif* when you have multiple mutually exclusive conditions to check.

```
if x > 0:  
    print("x is positive!")  
elif x < 0:  
    print("x is negative!")  
else:  
    print("x is zero!")
```



Elif in an *If* Statement

- Python evaluates the conditions from top to bottom. As soon as it finds a condition that is **true**, it executes that block and skips all remaining conditions.

```
score = 75

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
else:
    print("F")
```




Nesting *If* Statements

- It is possible to nest one if statement inside another
 - The indentation is key here

```
if x >= 0:  
    if x > 0:  
        print("x is positive!")  
    else:  
        print("x is zero!")  
else:  
    print("x is negative!")
```



If Expressions

- Recap: expressions and statements are different
 - Expressions return a value
 - Statements do not
- If expression is a short hand form of an if statement that returns a value
 - It is quite common to want to assign a specific value to a variable dependent on some condition

<result1> if <condition-is-met> else <result2>

a = 2

b = 330

print("A") if a > b else print("B")

status = "positive" if x > 0 else "not positive"



Assign a Value With *If ... Else*

- We can also use a one-line *if/else* to choose a value and assign it to a variable:

```
status = "positive" if x > 0 else "not positive"
```

- We can chain conditional expressions, but keep it short so it stays readable:

```
a = 330  
b = 330  
print("A") if a > b else print("=") if a == b else print("B")
```



Use Shorthand *If* ?

- Shorthand *if* statements and ternary operators should be used when:
 - The condition and actions are simple
 - It improves code readability
 - You want to make a quick assignment based on a condition



Match Statement

- Instead of writing many *if..else* statements, we can use the *match* statement
- The *match* statement selects one of many code blocks to be executed

```
match expression:
```

```
  case x:
```

```
    code block
```

```
  case y:
```

```
    code block
```

```
  case z:
```

```
    code block
```

This is how it works:

- The match expression is evaluated once.
- The value of the expression is compared with the values of each case.
- If there is a match, the associated block of code is executed.



Match Statement

```
day = 4
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case 4:
        print("Thursday")
    case 5:
        print("Friday")
    case 6:
        print("Saturday")
    case 7:
        print("Sunday")
```



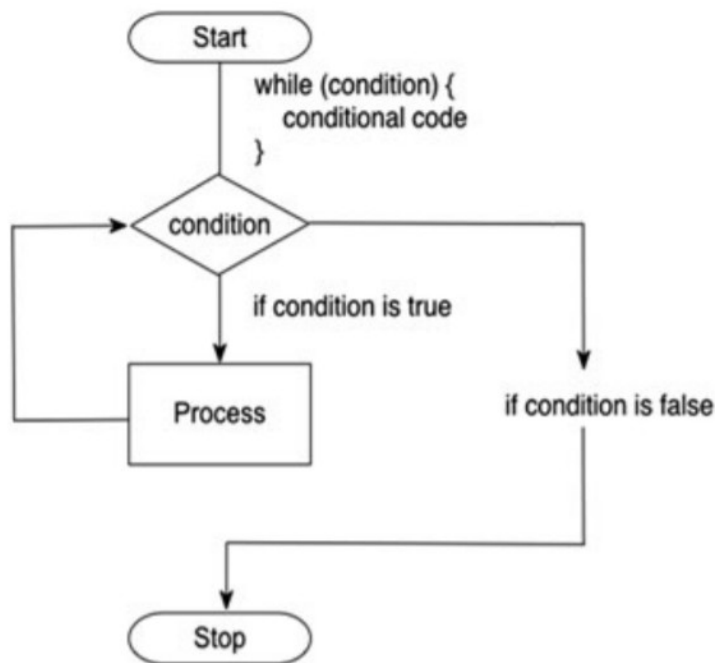
Iterative Statements

- The iterative statements are used to control the repeated execution of selected statements
 - *While*
 - *For*



While Loop

- With the *while* loop we can execute a set of statements as long as a condition is *true*.



*while <test-condition-is-true>:
statement or statements*

- *** Print *i* as long as *i* is less than 6:**

```
i = 1  
while i < 6:  
    print(i)  
    i += 1
```



While Loop

[Hunt 2019]

22

```
i = 1
while i < 6:
    print(i)
    i += 1
else:
    print("i is no longer less than 6")
```

Output?



For Loop

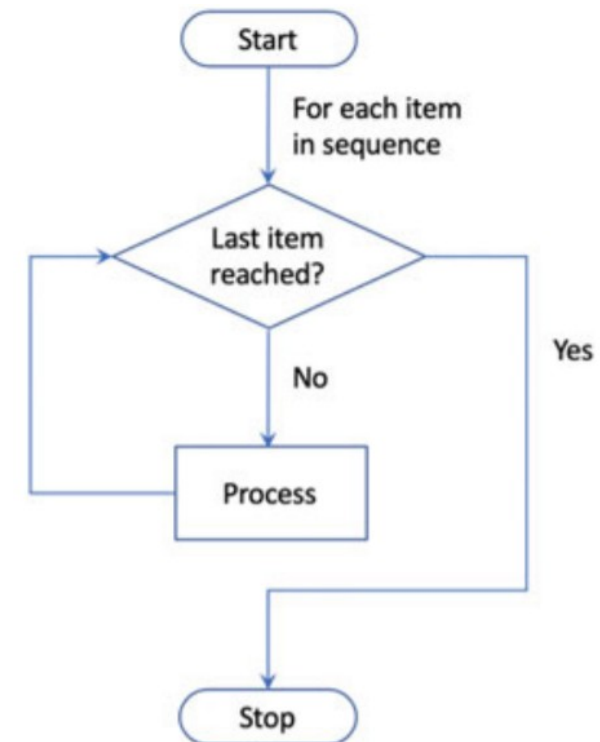
[Hunt 2019]

23

- A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).
- With the for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

```
for <variable-name> in <a-collection>:  
    statement(s)
```

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)
```





For Loop

```
for x in "banana":  
    print(x)
```

Output?



Range Function

[Hunt 2019]

https://www.w3schools.com/python/ref_func_range.asp

25

- To loop through a set of code a specified number of times, we can use the *range()* function,
- The *range()* function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number
- *range(10)* is the same as *range(0, 10, 1)*
 - [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
- Variable names should be meaningful in general
 - However, it is common to define loop variable names as *i*, *j*, etc.
 - Consider using these variable names in looping constructs
 - Avoid using them elsewhere

```
for i in range(10):  
    print(i)
```



Range Function

[Hunt 2019]

https://www.w3schools.com/python/ref_func_range.asp

26

```
for x in range(2, 30, 3):  
    print(x)
```

Output?

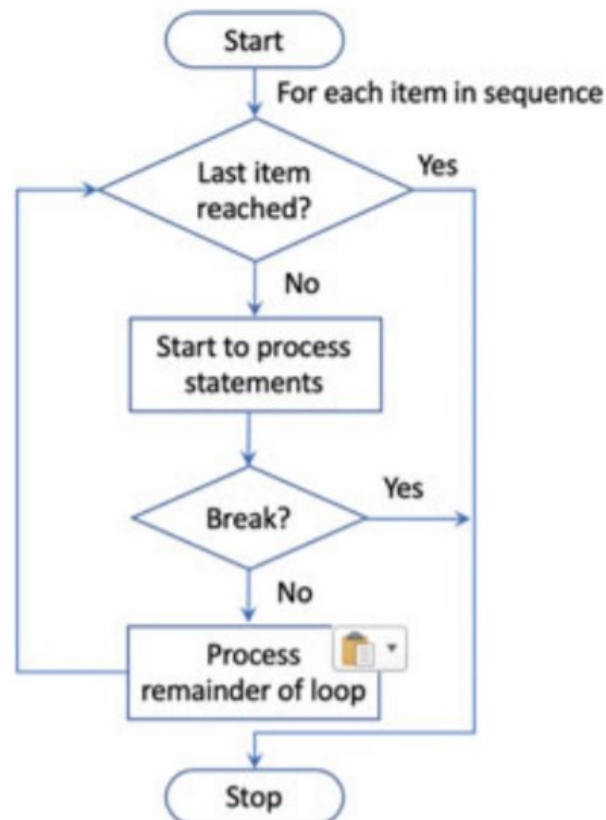


Break Loop Statement

[Hunt 2019]

27

- With the *break* statement we can stop the loop even if the loop condition is true:

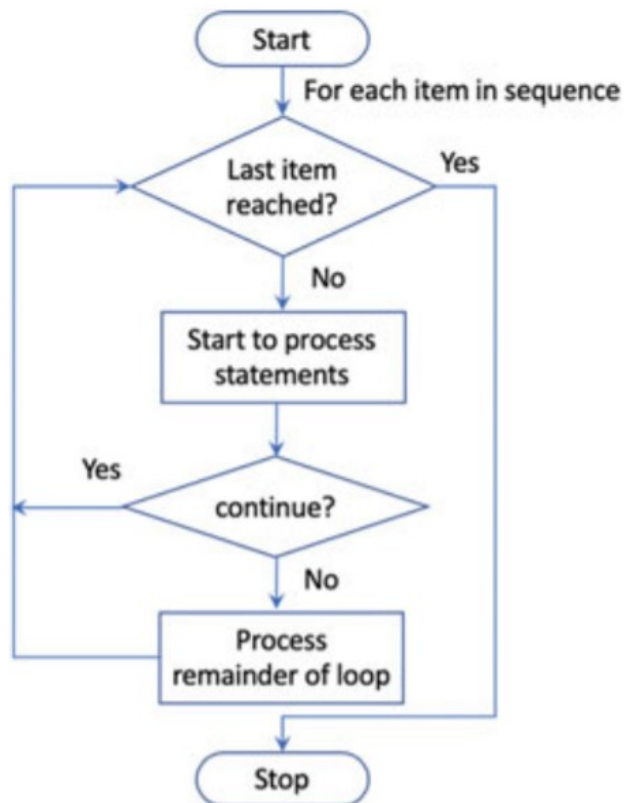


```
for i in range(10):  
    print(i)  
    if i == 5:  
        break
```



Continue Loop Statement

- With the *continue* statement we can stop the current iteration, and continue with the next:



```
for i in range(10):  
    if i == 5:  
        continue  
    print(i)
```




Pass Statement

- The ***pass*** statement is a ***null*** operation - nothing happens when it executes.
- For example, ***if*** statements cannot be empty, but if you for some reason have an ***if*** statement with no content, put in the ***pass*** statement to avoid getting an error.

```
age = 16

if age < 18:
    pass # TODO: Add underage logic later
else:
    print("Access granted")
```

```
for i in range(10):
    if i % 2 == 0:
        print(i)
    else:
        pass
```